TEXT-TO-SQL · SMALL LANGUAGE MODELS

A 1.5B model that writes SQL as well as a 7B

What I learned building, measuring and serving it. And three numbers I got wrong on the way.

71.5%	2,147	1.5B
EXECUTION ACCURACY	HELD-OUT QUESTIONS	PARAMETERS

Pradhyumna Holla

Full technical write-up in progress

THE PROBLEM

Ask a database a question in English

You have a database. Someone asks a question in plain English. Something has to turn that into SQL that actually runs and returns the right rows.

That is easy on a database you were trained on. This project does it on databases the model has never seen, at 1.5 billion parameters, which is small enough to serve without a GPU budget behind it.

2,147	206	1.5B
HELD-OUT QUESTIONS	UNSEEN DATABASES	PARAMETERS

THE ONE DECISION EVERYTHING RESTS ON

Measure by running the SQL, not by reading it

The obvious way to grade generated SQL is to compare its text against a known-correct query. That marks two identical answers wrong for writing their joins in a different order.

Instead: run both queries against the real database and compare the rows that come back. Two queries that look nothing alike but return the same answer are both correct.

One piece of code, doing four different jobs:

- **Scoring**
  how the model is graded, on databases it has never seen
- **Training reward**
  reward comes from the rows a query returns, not how it looks
- **Data filter**
  generated training data is kept only if it actually works
- **Answering**
  eight queries per question, grouped by the rows they return

WHAT I BUILT

End to end, not a notebook

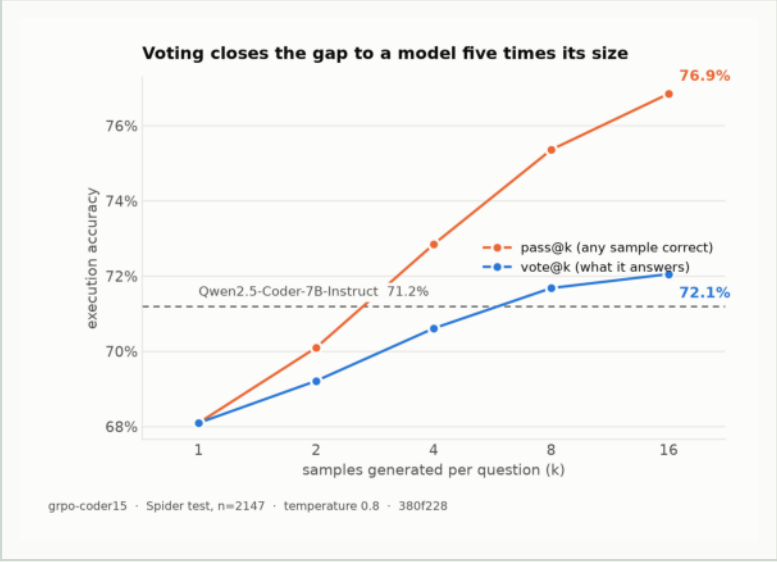
- **Start: Qwen2.5-Coder-1.5B**
  an open model from Alibaba, already trained on code. Nothing to do with SQL yet.
- **Teach it the task**
  supervised fine-tuning on 7,000 question-and-query pairs: show it the right answer and have it imitate.
- **Reinforcement learning (GRPO)**
  Group Relative Policy Optimisation: write several queries per question, run every one against the database, reinforce the ones that returned correct rows.
- **Answer eight times, not once**
  at question time it writes eight queries, all eight are run, and the answer is whichever result the majority produced.
- **Serve it over HTTP**
  a FastAPI endpoint returning the query, the rows, a confidence level and a timing breakdown.

All of it on one AWS g5.xlarge: a single NVIDIA A10G with 24GB of memory, 4 vCPUs, $1.006 an hour.

338	11	~$14
TESTS	GPU-HOURS MEASURING	TOTAL COMPUTE

THE RESULT

71.5%, level with a model five times larger

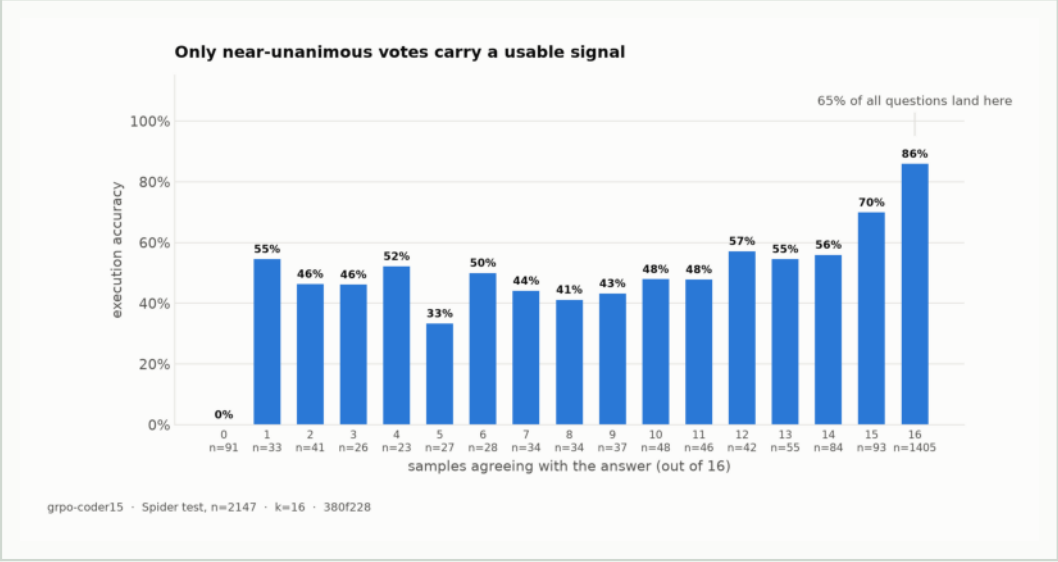


Blue is what the system actually replies with: ask it k times, run every query, answer with whichever result won the vote. Orange is whether ANY of those tries was right, which you only know by checking the answer key afterwards. Orange is the ceiling.

Blue flattens: 8 tries to 16 buys 0.4 points. Orange keeps rising. At 16 tries the model writes a correct query for 77% of questions and picks it only 72% of the time. That 5-point gap is right answers it produced and threw away.

A SIGNAL YOU GET FOR FREE

The model already knows when it is wrong

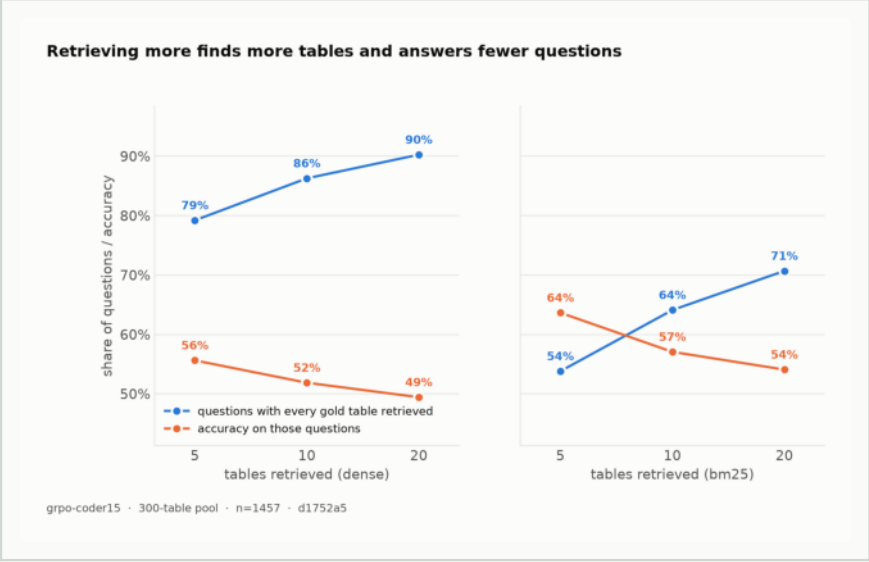


Each bar is one level of self-agreement. Far right is every sample agreeing with every other. Far left is none of them even producing a query that runs. Bar height is how often that group turned out to be correct.

When all sixteen agree it is right 86% of the time, covering 65% of questions. When they disagree it is near a coin flip. The middle bars jump around because the number under each is how many questions landed there, 20 to 80, far too few to read a trend into. Only the right-hand bar has enough behind it to trust.

THE FINDING I ALMOST MISSED

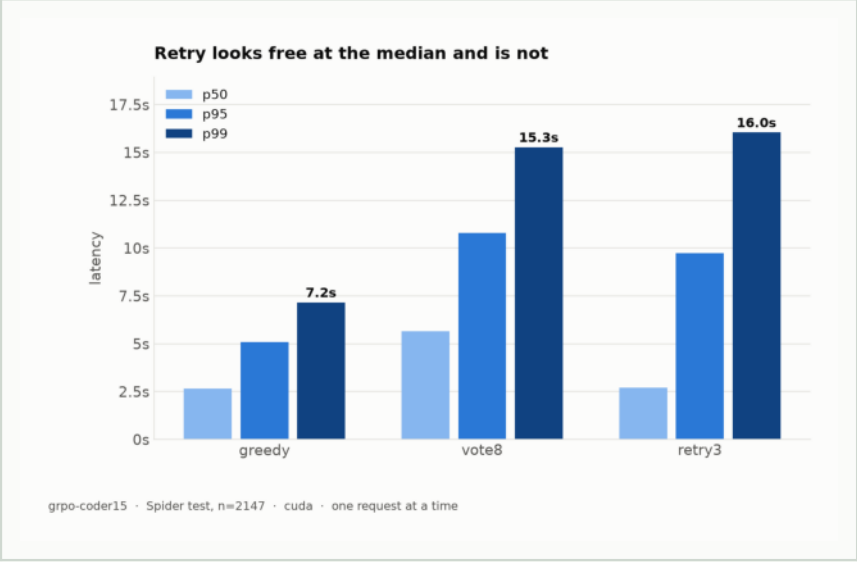
A flat number hiding two opposing forces



Real databases have hundreds of tables, so something must search for the relevant ones first. I gave it a 300-table haystack. Accuracy came back 44.3%, 45.2%, 44.8% when the search returned 5, 10, then 20 tables. Flat enough to conclude it does not matter.

It does. Blue is how often the search found every table the answer needed: it rises, because searching wider finds more. Orange is how often the model then got those same questions right: it falls, because the extra tables are distractions. They cancel.

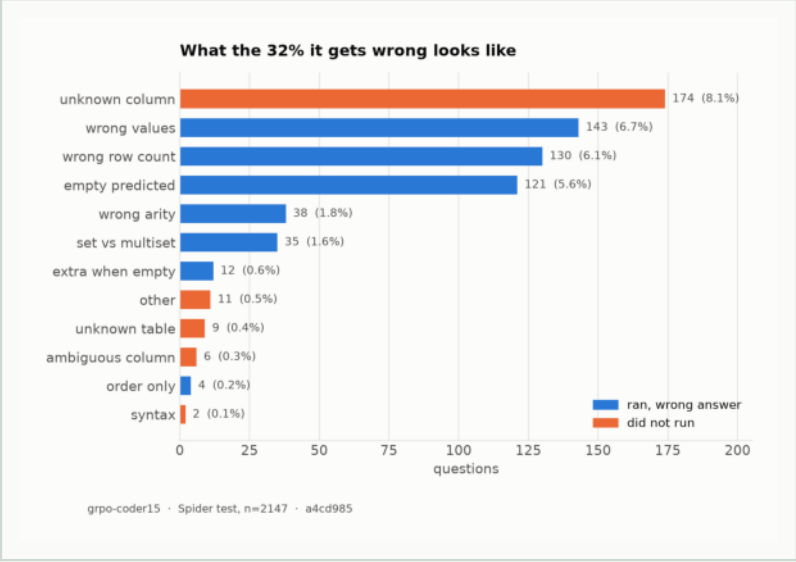
Accuracy reported next to its price



p50 is how long a typical question takes. p99 is the slowest one in every hundred. Retry here means: when a query hits a database error, show the model the error and let it try again.

Retry's typical question is as fast as not retrying at all, because most queries work first time and never enter the loop. But the ones that do retry are slow, and one question in a hundred takes 16 seconds. The average across everything is 3.8 seconds, which describes neither the fast case nor the slow one. Averages are how a technique looks free in a benchmark and hurts in production.

The largest failure is not reasoning



Every question it got wrong, sorted by what went wrong. Orange means the query crashed against the database. Blue means the query ran perfectly and handed back the wrong rows.

The single biggest failure is the model inventing a column name that does not exist in the database: 174 questions, 8.1% of the test set, bigger than any category of wrong answer. That is already 29% better than before reinforcement learning. So the next thing to fix is how the database structure is described to the model, not how the model reasons about it.

Three numbers I got wrong

01 **I said one question in a hundred took 28 seconds.**

I had measured that over only 100 questions, so it was simply the single slowest one I happened to see. Across all 2,147 it is 16 seconds, and what I concluded from 28 was wrong.

02 **I said searching for fewer tables would help.**

A guess, written down as though it were a result. Run properly at 5, 10 and 20 tables across two search methods, it does not help. Withdrawn.

03 **I said missing tables cost 9 points of accuracy.**

Table search fails two ways: it misses one the answer needs, or it finds them and buries them among irrelevant ones. Missing costs 6.7 points, not 9. Burying costs 16.5. I had it backwards.

All three are still in the write-up, beside the corrected versions. Anyone can report a benchmark number. The ones you had to withdraw are harder to fake.

NEXT

What I would build next

- **Train something to pick the answer.**

  Taking the majority vote throws away 4.8 points of accuracy the model already produced. I wrote two hand-made rules to choose better; one gained 0.1 points and the other lost 0.1. Choosing well needs a trained model, not a rule of thumb.

- **Train it on messy databases, not clean ones.**

  It learned on tidy, correct database descriptions and is then asked to work with cluttered ones full of irrelevant tables. Teaching it to ignore the clutter is a training problem, not a search problem.

- **Make it faster to serve.**

  2.7 seconds for a typical question is slow. Nothing in the current serving stack is optimised for inference speed, and there is an obvious library to swap in.

I am writing this up properly, with the methods and the numbers that did not make it here.

Follow if you want it when it lands.